```
import numpy as np
```

SESSION 4 — Array Operations

Array operations are the core strength of NumPy.

Why do we need array operations?

Because in scientific computing and machine learning we frequently perform:

- Element-wise arithmetic
- Comparisons
- Logical filtering
- Mathematical transformations

NumPy allows these operations without explicit loops. This is called vectorization.

---

1. Arithmetic Operations

Operators supported:

- `+  Addition`
- `-  Subtraction`
- `*  Multiplication`
- / Division
- // Floor Division
- ** Power
- % Modulus

These operate element-wise when applied to arrays.

```
import numpy as np

a = np.array([10, 20, 30])
b = np.array([2, 4, 5])

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Floor Division:", a // b)
print("Power:", a ** 2)
print("Modulus:", a % b)
```

```
Addition: [12 24 35]
Subtraction: [ 8 16 25]
Multiplication: [ 20  80 150]
Division: [5. 5. 6.]
Floor Division: [5 5 6]
Power: [100 400 900]
Modulus: [0 0 0]
```

Why these operators exist:

- Enable vectorized computation
- Avoid explicit loops
- Faster execution due to C backend
- Memory efficient

---

2. Comparison Operators

Used for element-wise comparison.

== Equal
!= Not equal

> Greater than
> < Less than
> = Greater than or equal
> <= Less than or equal

Output is a Boolean array.

```
a = np.array([5, 10, 15])

print("Greater than 8:", a[a > 8])
print("Equal to 10:", a == 10)
```

```
Greater than 8: [False  True  True]
Equal to 10: [False  True False]
```

These Boolean arrays are used for filtering.

```
print("Elements greater than 8:", a[a > 8])
```

```
Elements greater than 8: [10 15]
```

3. Logical Operations

Used when combining multiple conditions.

Functions:

np.logical_and(condition1, condition2)
np.logical_or(condition1, condition2)
np.logical_not(condition)

Why these exist:

- Combine Boolean conditions
- Enable complex filtering
- Used heavily in data analysis

```
a = np.array([5, 10, 15, 20])

cond1 = a > 8
cond2 = a < 18

print("Logical AND:", np.logical_and(cond1, cond2))
print("Filtered Result:", a[np.logical_and(cond1, cond2)])

print("Logical OR:", np.logical_or(cond1, cond2))
print("Logical NOT:", np.logical_not(cond1))
```

```
Logical AND: [False  True  True False]
Filtered Result: [10 15]
Logical OR: [ True  True  True  True]
Logical NOT: [ True False False False]
```

4. Broadcasting

Broadcasting allows operations between arrays of different shapes.

Rule: NumPy automatically expands dimensions when compatible.

Example:

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
```

```
b = np.array([10, 20, 30])

print("Broadcasted Addition:\n", a + b)
```

```
Broadcasted Addition:
 [[11 22 33]
 [14 25 36]]
```

Why Broadcasting is Powerful:

- Avoids manual reshaping
- Reduces memory duplication
- Speeds up computations

---

5. Element-wise vs Aggregate Operations

Element-wise: Operation applied to each element individually.

Aggregate: Operation reduces array to single value.

```
arr = np.array([1, 2, 3, 4])

print("Element-wise square:", arr ** 2)
print("Aggregate sum:", np.sum(arr))
```

```
Element-wise square: [ 1  4  9 16]
Aggregate sum: 10
```

SESSION 5 — Mathematical & Statistical Functions

NumPy provides optimized mathematical functions.

Why separate functions exist when operators already work?

Because:

- Functions allow explicit usage
- Useful in functional programming
- More readable in complex expressions
- Support broadcasting and advanced parameters

1. Basic Mathematical Functions

Functions:

- np.add()
- np.subtract()
- np.multiply()
- np.divide()
- np.power()
- np.mod()
- np.sqrt()
- np.exp()
- np.log()
- np.log10()

```
a = np.array([4, 9, 16])
b = np.array([2, 3, 4])

print("Add:", np.add(a, b))
print("Subtract:", np.subtract(a, b))
print("Multiply:", np.multiply(a, b))
print("Divide:", np.divide(a, b))
print("Power:", np.power(a, 2))
```

```
print("Mod:", np.mod(a, b))

print("Square root:", np.sqrt(a))
print("Exponential:", np.exp(1))
print("Natural log:", np.log(a))
print("Log base 10:", np.log10(a))
```

```
Add: [ 6 12 20]
Subtract: [ 2  6 12]
Multiply: [ 8 27 64]
Divide: [2. 3. 4.]
Power: [ 16  81 256]
Mod: [0 0 0]
Square root: [2. 3. 4.]
Exponential: 2.718281828459045
Natural log: [1.38629436 2.19722458 2.77258872]
Log base 10: [0.60205999 0.95424251 1.20411998]
```

Functions:

np.round()
np.floor()
np.ceil()

Used in financial calculations and data cleaning.

```
arr = np.array([1.4, 2.6, 3.2])

print("Round:", np.round(arr))
print("Floor:", np.floor(arr))
print("Ceil:", np.ceil(arr))
```

```
Round: [1. 3. 3.]
Floor: [1. 2. 3.]
Ceil: [2. 3. 4.]
```

Statistical operations reduce arrays.

Functions:

np.mean()
np.median()
np.std()
np.var()
np.min()
np.max()
np.sum()
np.cumsum()
np.prod()
np.percentile()

Why these exist:

- Data analysis
- Machine learning preprocessing
- Performance metrics

```
data = np.array([10, 40, 30, 10, 50])

print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Standard Deviation:", np.std(data))
print("Variance:", np.var(data))
print("Minimum:", np.min(data))
print("Maximum:", np.max(data))
print("Sum:", np.sum(data))
print("Cumulative Sum:", np.cumsum(data))
print("Product:", np.prod(data))
print("90th Percentile:", np.percentile(data, 80))
```

```
Mean: 30.0
Median: 30.0
Standard Deviation: 14.142135623730951
```

```
Variance: 200.0
Minimum: 10
Maximum: 50
Sum: 150
Cumulative Sum: [ 10  30  60 100 150]
Product: 12000000
90th Percentile: 42.0
```

NumPy Practice Set — Session 4 & 5

(Array Operations + Mathematical & Statistical Functions)

EASY LEVEL

1. Create two arrays: a = [5, 10, 15] b = [1, 2, 3] Perform element-wise addition and multiplication.
2. Given array: arr = [4, 8, 12, 16] Extract elements greater than 10.
3. Create a 2×3 array of ones and add 5 to every element using broadcasting.
4. Given array: arr = [10, 20, 30, 40] Compute:
   - Mean
   - Sum
   - Maximum
5. Convert 45 degrees into radians and compute its sine value.

MEDIUM LEVEL

6. Create two arrays: a = [2, 4, 6, 8] b = [1, 2, 3, 4] Perform floor division and modulus operations.
7. Generate a 3×3 random integer matrix between 1 and 20. Find its standard deviation and variance.
8. Create an array from 1 to 9 and reshape it into 3×3. Compute cumulative sum (cumsum).
9. Create array: arr = [-3, -1, 0, 1, 3] Apply:
   - Square
   - Square root (only valid values)
10. Using logical operations: Given array: arr = [5, 10, 15, 20, 25] Extract values between 10 and 20 inclusive.

HARD LEVEL

11. Create a 3×4 matrix using np.arange(). Add a 1D array [10, 20, 30, 40] using broadcasting.
12. Generate 1000 random numbers from normal distribution. Compute:
    - Mean
    - Standard deviation
    - 95th percentile
13. Create two matrices: A (2×3) and B (3×2). Transpose A and verify its shape.
14. Create array: arr = [1, 2, 3, 4] Compute:
    - Exponential of each element
    - Natural log
    - Log base 10
15. Create array: arr = [1.2, 2.5, 3.7, 4.1] Apply:
    - round()
    - floor()
    - ceil() Explain differences based on output.

```
import numpy as np

# -------------------------------
# EASY LEVEL SOLUTIONS
# -------------------------------
```

```
# 1
a = np.array([5, 10, 15])
b = np.array([1, 2, 3])

print("Q1 Addition:", a + b)          # Element-wise addition
print("Q1 Multiplication:", a * b)    # Element-wise multiplication


# 2
arr = np.array([4, 8, 12, 16])
print("Q2 Elements > 10:", arr[arr > 10])  # Boolean filtering


# 3
arr = np.ones((2, 3))
print("Q3 Broadcasting Result:\n", arr + 5)  # Scalar broadcasting


# 4
arr = np.array([10, 20, 30, 40])
print("Q4 Mean:", np.mean(arr))
print("Q4 Sum:", np.sum(arr))
print("Q4 Max:", np.max(arr))


# 5
angle_rad = np.radians(45)            # Convert degrees to radians
print("Q5 Sine of 45 degrees:", np.sin(angle_rad))


# -------------------------------
# MEDIUM LEVEL SOLUTIONS
# -------------------------------

# 6
a = np.array([2, 4, 6, 8])
b = np.array([1, 2, 3, 4])
print("Q6 Floor Division:", a // b)
print("Q6 Modulus:", a % b)


# 7
np.random.seed(0)
matrix = np.random.randint(1, 21, (3, 3))
print("Q7 Matrix:\n", matrix)
print("Q7 Std:", np.std(matrix))
print("Q7 Variance:", np.var(matrix))


# 8
arr = np.arange(1, 10).reshape(3, 3)
print("Q8 Matrix:\n", arr)
print("Q8 Cumulative Sum:", np.cumsum(arr))


# 9
arr = np.array([-3, -1, 0, 1, 3])
print("Q9 Square:", np.power(arr, 2))

# Square root only valid for non-negative numbers
print("Q9 Square Root (valid only):", np.sqrt(arr[arr >= 0]))


# 10
arr = np.array([5, 10, 15, 20, 25])
condition = np.logical_and(arr >= 10, arr <= 20)
print("Q10 Between 10 and 20:", arr[condition])


# -------------------------------
# HARD LEVEL SOLUTIONS
# -------------------------------

# 11
matrix = np.arange(12).reshape(3, 4)
vector = np.array([10, 20, 30, 40])
```

```
print("Q11 Original Matrix:\n", matrix)
print("Q11 After Broadcasting:\n", matrix + vector)


# 12
np.random.seed(1)
data = np.random.randn(1000)
print("Q12 Mean:", np.mean(data))
print("Q12 Std:", np.std(data))
print("Q12 95th Percentile:", np.percentile(data, 95))


# 13
A = np.arange(6).reshape(2, 3)
B = np.arange(6).reshape(3, 2)
print("Q13 A:\n", A)
print("Q13 Transposed A:\n", A.T)
print("Q13 Shape of A.T:", A.T.shape)


# 14
arr = np.array([1, 2, 3, 4])
print("Q14 Exponential:", np.exp(arr))
print("Q14 Natural Log:", np.log(arr))
print("Q14 Log Base 10:", np.log10(arr))


# 15
arr = np.array([1.2, 2.5, 3.7, 4.1])
print("Q15 Round:", np.round(arr))    # Nearest integer
print("Q15 Floor:", np.floor(arr))    # Round down
print("Q15 Ceil:", np.ceil(arr))      # Round up
```

```
Q1 Addition: [ 6 12 18]
Q1 Multiplication: [ 5 20 45]
Q2 Elements > 10: [12 16]
Q3 Broadcasting Result:
 [[6. 6. 6.]
 [6. 6. 6.]]
Q4 Mean: 25.0
Q4 Sum: 100
Q4 Max: 40
Q5 Sine of 45 degrees: 0.7071067811865475
Q6 Floor Division: [2 2 2 2]
Q6 Modulus: [0 0 0 0]
Q7 Matrix:
 [[13 16  1]
 [ 4  4  8]
 [10 20 19]]
Q7 Std: 6.499762578759852
Q7 Variance: 42.24691358024692
Q8 Matrix:
 [[1 2 3]
 [4 5 6]
 [7 8 9]]
Q8 Cumulative Sum: [ 1  3  6 10 15 21 28 36 45]
Q9 Square: [9 1 0 1 9]
Q9 Square Root (valid only): [0.         1.         1.73205081]
Q10 Between 10 and 20: [10 15 20]
Q11 Original Matrix:
 [[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
Q11 After Broadcasting:
 [[10 21 32 43]
 [14 25 36 47]
 [18 29 40 51]]
Q12 Mean: 0.03881247615960185
Q12 Std: 0.9810041339322116
Q12 95th Percentile: 1.6278527911730007
Q13 A:
 [[0 1 2]
 [3 4 5]]
Q13 Transposed A:
 [[0 3]
 [1 4]
 [2 5]]
Q13 Shape of A.T: (3, 2)
Q14 Exponential: [ 2.71828183  7.3890561  20.08553692 54.59815003]
Q14 Natural Log: [0.         0.69314718 1.09861229 1.38629436]
Q14 Log Base 10: [0.         0.30103    0.47712125 0.60205999]
```

```
Q15 Round: [1. 2. 4. 4.]
Q15 Floor: [1. 2. 3. 4.]
Q15 Ceil: [2. 3. 4. 5.]
```

SESSION 6 — Indexing & Slicing in NumPy

Indexing and slicing are fundamental for:

- Accessing specific elements
- Extracting subsets of data
- Filtering datasets
- Modifying portions of arrays

Without indexing and slicing, data analysis is not practical.

---

1. Basic Indexing (1D Arrays)

Why It Exists

To access individual elements from an array.

Syntax

arr[index]

Index starts from 0.

```
import numpy as np

arr = np.array([10, 20, 30, 40])

print("Element at index 0:", arr[0])
print("Element at index 2:", arr[2])
```

```
Element at index 0: 10
Element at index 2: 30
```

2. Negative Indexing

Why It Exists

To access elements from the end of the array.

Syntax

arr[-1] → last element
arr[-2] → second last element

```
print("Last element:", arr[-1])
print("Second last:", arr[-2])
```

```
Last element: 40
Second last: 30
```

3. Basic Indexing (2D Arrays)

Why It Exists

To access elements in matrices.

Syntax

arr[row_index, column_index]

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

print("Element at (1,2):", matrix[1, 1])
```

```
Element at (1,2): 5
```

4. Slicing

Why It Exists

To extract subarrays.

Syntax

arr[start:stop:step]

- start → inclusive
- stop → exclusive
- step → interval

```
arr = np.array([0, 10, 20, 30, 40, 50])

print("Slice 1:4:", arr[1:4])
print("Slice with step 2:", arr[0:6:2])
```

```
Slice 1:4: [10 20 30]
Slice with step 2: [ 0 20 40]
```

5. 2D Slicing

Syntax:

arr[row_start:row_end, col_start:col_end]

```
print("Submatrix:\n", matrix[0:2, 1:3])
```

```
Submatrix:
 [[2 3]
 [5 6]]
```

7. Boolean Indexing

Why It Exists

To filter arrays based on conditions.

Syntax

arr[condition]

```
arr = np.array([5, 10, 15, 20])

print("Elements > 10:",arr > 10)
print("Elements > 10:", arr[arr > 10])
```

```
Elements > 10: [False False  True  True]
Elements > 10: [15 20]
```

8. Indexing with np.where()

Why It Exists

To get indices where condition is True.

Syntax

np.where(condition)

```
arr = np.array([5, 10, 15, 20])

indices = np.where(arr > 10)
print("Indices:", indices)
print("Values:", arr[indices])
```

```
Indices: (array([2, 3]),)
Values: [15 20]
```

```
arr = np.array([5, 10, 15, 20])

# print(np.where(arr > 10))
print(arr[np.where(arr > 10)])
# print("Indices:", indices)
# print("Values:", arr[indices])
```

```
[15 20]
```

Practice Questions — Session 6 (Indexing & Slicing)

SESSION 6

1.

Create a 1D NumPy array: arr = [10, 20, 30, 40, 50]

Print:

- First element
- Last element using negative indexing

2.

Given array: arr = [5, 10, 15, 20, 25, 30]

Extract elements from index 1 to 4 using slicing.

3.

Given array: arr = [0, 2, 4, 6, 8, 10]

Extract every second element using step slicing.

4.

Create a 2D array: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

Print the element located at: Row 2, Column 1 (using proper indexing).

5.

Using the same 2D array from Question 4, Extract the first two rows.

6.

Using the same 2D array, Extract the submatrix: [[2, 3], [5, 6]]

7.

Create array: arr = [10, 20, 30, 40, 50]

Using fancy indexing, extract:

- 1st element
- 3rd element
- 5th element

8.

Create array: arr = [5, 10, 15, 20, 25]

Using boolean indexing, extract elements greater than 12.

9.

Given array: arr = [3, 6, 9, 12, 15]

Use np.where() to find indices where values are greater than 8.

10.

Create array: arr = [1, 2, 3, 4, 5]

Reverse the array using slicing.

REVISION QUESTIONS (Previous Sessions — 2 Questions)

11. (Array Operations)

Create arrays: a = [2, 4, 6] b = [1, 2, 3]

Perform:

- Element-wise multiplication
- Element-wise power (a squared)

---

12. (Mathematical Functions)

Create array: arr = [4, 9, 16, 25]

Compute:

- Square root
- Mean
- Maximum value

```
import numpy as np

# -----------------------------------
# SESSION 6 SOLUTIONS
# -----------------------------------

# 1
arr = np.array([10, 20, 30, 40, 50])
print("Q1 First element:", arr[0])          # Index 0
print("Q1 Last element:", arr[-1])          # Negative indexing


# 2
arr = np.array([5, 10, 15, 20, 25, 30])
print("Q2 Slice index 1 to 4:", arr[1:5])   # Stop index excluded


# 3
arr = np.array([0, 2, 4, 6, 8, 10])
print("Q3 Every second element:", arr[::2]) # Step slicing



# 4
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

print("Q4 Element at Row 2, Column 1:", matrix[2, 1])
# Row index 2 → third row
```

```
# Column index 1 → second column


# 5
print("Q5 First two rows:\n", matrix[0:2, :])
# All columns of first two rows


# 6
print("Q6 Submatrix:\n", matrix[0:2, 1:3])
print("Q6 Submatrix:\n", matrix[1:3, 0:2])
# Rows 0 to 1, Columns 1 to 2


# 7
arr = np.array([10, 20, 30, 40, 50])
print("Q7 Fancy indexing:", arr[[0, 2, 4]])


# 8
arr = np.array([5, 10, 15, 20, 25])
print("Q8 Boolean indexing (>12):", arr[arr > 12])


# 9
arr = np.array([3, 6, 9, 12, 15])
indices = np.where(arr > 8)
print("Q9 Indices where > 8:", indices)
print("Q9 Values where > 8:", arr[indices])


# 10
arr = np.array([1, 2, 3, 4, 5])
print("Q10 Reversed array:", arr[::-1])
# Step = -1 reverses array


# ----------------------------------
# REVISION QUESTIONS (Previous Sessions)
# ----------------------------------

# 11
a = np.array([2, 4, 6])
b = np.array([1, 2, 3])

print("Q11 Multiplication:", a * b)   # Element-wise multiplication
print("Q11 Power (a squared):", a ** 2)


# 12
arr = np.array([4, 9, 16, 25])

print("Q12 Square root:", np.sqrt(arr))
print("Q12 Mean:", np.mean(arr))
print("Q12 Maximum:", np.max(arr))
```

```
Q1 First element: 10
Q1 Last element: 50
Q2 Slice index 1 to 4: [10 15 20 25]
Q3 Every second element: [0 4 8]
Q4 Element at Row 2, Column 1: 8
Q5 First two rows:
 [[1 2 3]
 [4 5 6]]
Q6 Submatrix:
 [[2 3]
 [5 6]]
Q6 Submatrix:
 [[4 5]
 [7 8]]
Q7 Fancy indexing: [10 30 50]
Q8 Boolean indexing (>12): [15 20 25]
Q9 Indices where > 8: (array([2, 3, 4]),)
Q9 Values where > 8: [ 9 12 15]
Q10 Reversed array: [5 4 3 2 1]
Q11 Multiplication: [ 2  8 18]
Q11 Power (a squared): [ 4 16 36]
Q12 Square root: [2. 3. 4. 5.]
Q12 Mean: 13.5
```

```
Q12 Maximum: 25
```

Start coding or generate with AI.